Software made for you

EG DOCTRANS

TECHNICAL DOCUMENTATION

Usecase: Document Translation with Embedded Image Text Translation
Created on: 27-12-2024
Prepared for: EG
Prepared by: CTO-Team

Table of Contents

1. Overview 0
   1.1 Problem Statement 0
   1.2 Proposed Solution 0
   1.3 Solution scope 0
   1.4 Inclusion/Exclusion table 2
2. Cloud architecture 3
3. Technical architecture 5
   3.1 Document Translation 5
4. Data management 7
   4.1 Input Data Handling 7
   4.2 Intermediate Data Management 8
      4.2.1 Text Extraction: 8
      4.2.2 Structured Information: 8
      4.2.3 Data Processing: 8
   4.3 Translation Data Management 8
      4.3.1 Translation Input: 8
      4.3.2 Translation output 8
   4.4 Metadata Management 8
      4.4.1 Database Usage: 8
   4.5 Notifications and Delivery 9
      4.5.1 Completion Notifications: 9
      4.5.2 Final Outputs: 9
5. Folders and code modules 10
   5.1 React Frontend folder 10
   5.2 Backend folder 11
6. Example outputs 13
7. Cloud Infrastructure 14
   7.1 Compute 14
   7.2 Storage 14
   7.3 Networking 15
8. Deployment process 16

8.1 Prerequisites 16
8.2 Cloud Deployment Steps 17
8.3 CI/CD Pipeline Using JFrog Artifactory and AWS ECR 17
9. Debugging and troubleshooting 19
10. Monitoring and Maintenance 21
10.1 Azure cloud costing 22
11. UI Wireframe 23
11.1 User Flow: 23
12. Deployed links 25
13. Contact information 26

1. Overview

1.1 Problem Statement

The organization, EG, requires an efficient document translation solution to address internal communication challenges across multilingual teams. The solution must handle diverse document formats, ensure accurate translations with custom terminologies, preserve original structures, and comply with data security standards. It should integrate seamlessly into existing workflows, support scalable processing for high volumes, and provide robust error handling for a smooth user experience.

1.2 Proposed Solution

The proposed solution for EG's document translation challenges involves leveraging Azure Cognitive Services for accurate, scalable translations while maintaining document structure and applying custom glossaries for domain-specific terminology. Azure Blob Storage will ensure secure and efficient file management, integrating seamlessly with a streamlined workflow that automates document upload, validation, translation, and download processes. A user-friendly interface will facilitate easy interactions, with features like drag-and-drop uploads, real-time status tracking, and compliance confirmations to address data security concerns. The backend, built using FastAPI, will support asynchronous processing for scalability and robust performance, complemented by comprehensive error handling and email notifications. This solution ensures accurate translations, secure operations, and an enhanced user experience tailored to EG's needs.

1.3 Solution scope

1. **Target Users:** The application is designed for EG employees across various business units, providing multilingual document translation services to enhance internal communication and operational efficiency.
2. **Input Data:** The application supports the following file formats for input data:
    - **PDF** (.pdf)
    - **PowerPoint** (.pptx)
    - **Word Document** (.docx)

- **Images** (.png /.jpeg)

3. **Shared Architecture Design:** The application is built on a robust architecture that includes:
    - **Azure Blob Storage**: Centralized storage for managing input and translated files securely and efficiently.
    - **PostgreSQL Database**: For tracking file metadata, user actions, and translation status.
    - **FastAPI Framework**: Ensuring scalable, high-performance backend API services.
4. **Translation Services:** The solution integrates with Azure Cognitive Services to provide accurate translations while preserving original document structure. Additionally, custom glossaries can be applied to ensure domain-specific terminology is maintained.
5. **Language Support:** The application can process input data in any language and translate it into a wide range of target languages. It supports the addition of custom domain-specific vocabulary to improve translation accuracy.
6. **Image Translation:**Image-based text within supported document formats, such as PDFs and Word documents, can be extracted using OCR technology and translated into the target language.
7. **Compliance and Security:**
    - Users must confirm that input documents do not contain Personally Identifiable Information (PII) before initiating translation.
    - The application ensures compliance with EU GDPR standards by filtering out sensitive data during processing and providing clear compliance warnings in the interface.
8. **Notification System:** Users receive email notifications upon translation completion, including a download link for the translated document.
9. **Data Handling and Access:** The application provides secure authentication and authorization mechanisms, ensuring that only authorized users can access uploaded and translated files.

10. **Scalability and Performance**: The application supports asynchronous processing to handle large file sizes and high translation volumes, ensuring minimal delays and optimal user experience.
11. **Error Handling and Logging**: Comprehensive error handling mechanisms ensure that users are informed of issues like invalid formats or oversized files. Detailed logs track user interactions and system performance for troubleshooting and improvements.

1.4 Inclusion/Exclusion table

The section provides a clear outline for each of the feature and clarifies what is not in-scope and de-prioritized out of the current version of the application.

Feature	Inclusion	Exclusion	De-prioritized
Login and Authentication	OAuth-based login and access through a web link.	Offline authentication.	Advanced multi-factor authentication.
File Upload	Support for .docx and .pdf formats with size ≤ 40 MB.	Other file formats and sizes > 40 MB.	Real-time uploads for high-volume files.
Language Selection	Supports Polish, Norwegian, Finnish, Danish, Swedish, and English.	Selection of multiple target languages in one session.	Expansion to additional languages.
Glossary	Optional glossary support with up to 2000 word pairs for domain-specific translations.	Glossary support for images.	Multiple glossary files or exceeding the 2000-word pair limit.
GDPR Compliance	Popup confirmation for sensitive data before proceeding with translation.	Automated GDPR checks or encryption of sensitive data pre-translation.	Advanced AI-driven GDPR compliance scanning.
Translation Process	Asynchronous translation with	Immediate real-time	Enhanced notifications with progress tracking.

	email notification on completion.	translation feedback.	
Download Translated Files	Files available for download post-completion; one-time download.	Permanent availability of translated files for re-download.	Multi-format download options.

2. Cloud architecture

1. **Azure OCR:** Azure OCR (Optical Character Recognition) is a robust service for extracting text from images and scanned documents, transforming it into machine-readable formats. It supports both printed and handwritten text, offering compatibility with various layouts, fonts, and multiple languages. Azure OCR ensures high accuracy in text extraction while preserving the original document's structure, such as paragraph alignment, tables, and indentation. This capability makes it essential for digitizing physical documents like forms, business cards, and contracts, reducing manual data entry efforts. Azure OCR integrates seamlessly with other Azure services, enabling automated workflows for document management, translation, and archival. Additionally, it supports complex use cases like extracting content from low-resolution or distorted images through advanced machine learning algorithms.

2. **Azure Document Intelligence:** Azure Document Intelligence is a powerful tool for processing and extracting structured data from complex documents like invoices, receipts, contracts, and application forms. Leveraging AI-powered prebuilt models and customizable templates, the service identifies and extracts key-value pairs, tables, and important data elements. These are organized into structured formats like JSON or XML for downstream processing. This service minimizes manual intervention, streamlining workflows for data entry, invoice processing, and compliance checks. Azure Document Intelligence is also equipped with capabilities like detecting signatures, logos, and handwritten annotations, making

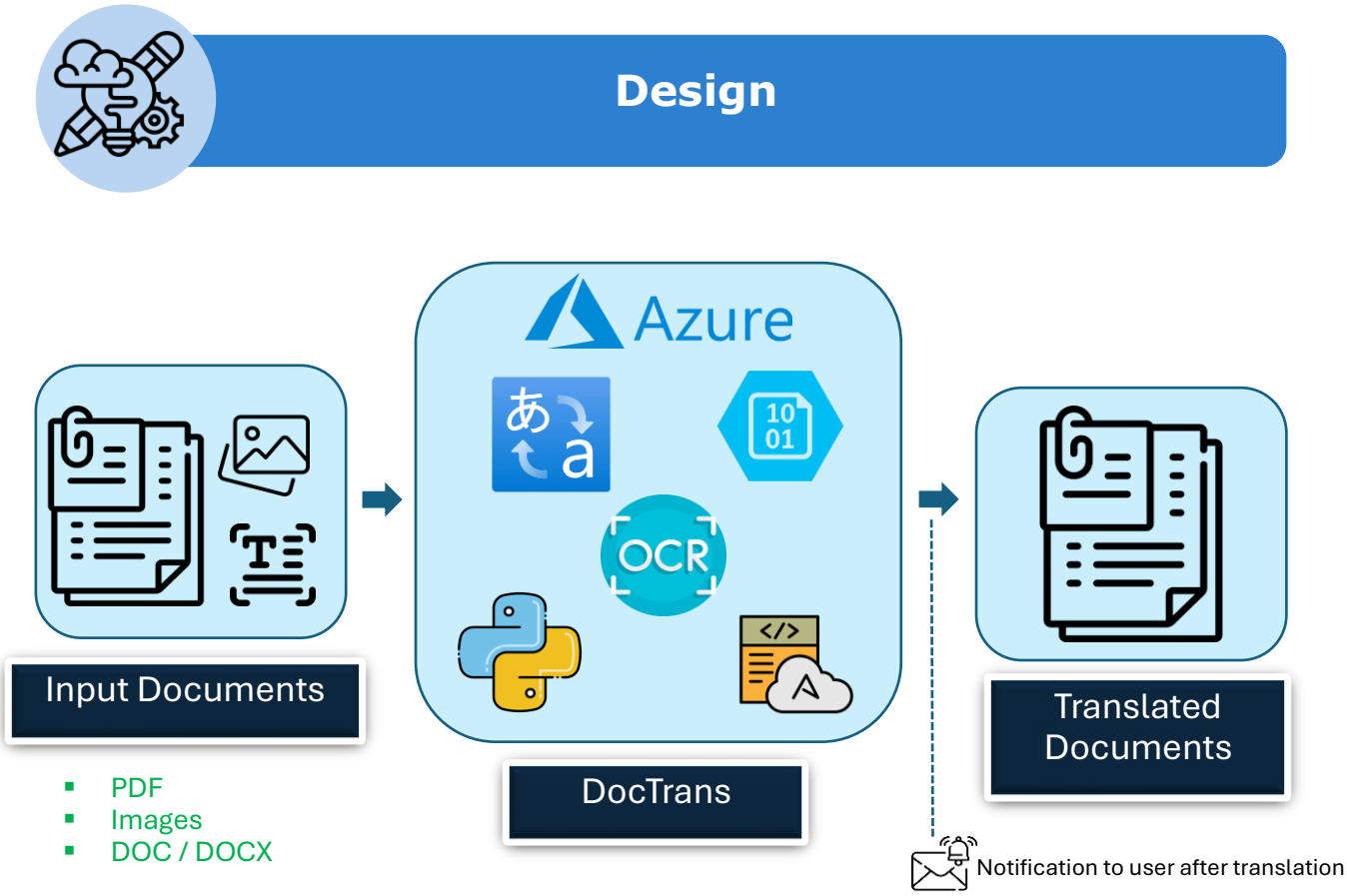
it versatile for various industries. It integrates with data analytics and reporting tools to enhance decision-making and operational efficiency.

3. **Azure Blob Storage:** Azure Blob Storage is a highly scalable and secure object storage solution for managing unstructured data, such as documents, images, videos, and backups. In this solution, all data is encrypted both at rest and during transit to ensure security and compliance. Each blob in the storage is tagged with metadata parameters, facilitating an event-driven architecture where metadata triggers actions, such as notifying downstream services or starting workflows. Blob Storage supports tiered storage options (hot, cool, and archive) to optimize costs based on access patterns. It integrates natively with Azure Event Grid, enabling real-time notifications and automation for operations like document processing and translation.

4. **Azure Text Translation:** Azure Text Translation is a comprehensive language translation service supporting more than 100 languages, designed for real-time or batch translation. It ensures the preservation of the original text's context and tone, making it suitable for technical documents, user-generated content, and corporate communications. Businesses can customize translations using glossaries to enforce domain-specific terminology consistency. The service integrates seamlessly with Azure Cognitive Services, enabling workflows such as multilingual chatbot responses, real-time subtitle generation, and international document handling. Its advanced machine translation models continuously improve through AI training, ensuring high accuracy and reliability. Azure Text Translation is scalable, catering to enterprises with large volumes of multilingual text processing requirements.

3. Technical architecture

3.1 Document Translation



1. **Input Layer :** The input layer acts as the first point of interaction for users, enabling them to upload documents via a web application or API. It supports various file formats, including PDFs, images (JPEG, PNG, TIFF), and others, ensuring flexibility for diverse use cases. Uploaded files are temporarily stored in Azure Blob Storage, a highly scalable and secure repository optimized for large-scale document management. This layer performs file validation, checking formats, sizes, and compliance with preconfigured criteria. By ensuring compatibility, the input layer guarantees that only valid documents proceed to subsequent processing stages. Additionally, user-friendly interfaces such as drag-and-drop upload and progress indicators enhance the user experience.

2. **Preprocessing Layer:** The preprocessing layer utilizes Azure OCR to extract textual data from uploaded documents, converting it into structured JSON format. This includes not only the text but also its layout information, such as bounding boxes, regions, and hierarchies. Advanced capabilities enable it to recognize multi-language content, handwritten text, and even distorted images, ensuring versatility in text extraction.
3. **Data Extraction Layer:** Azure Document Intelligence is the cornerstone of the data extraction layer, providing robust tools to process OCR outputs. It leverages prebuilt models for specific document types like invoices, receipts, and contracts, while also allowing customization for unique templates. This layer extracts actionable insights, such as customer details, invoice numbers, payment terms, and line-item data, from unstructured content. By organizing this information into structured formats (e.g., JSON, XML), it enables seamless integration with other systems like CRMs, ERPs, or analytics platforms. This automation eliminates repetitive manual tasks, reduces errors, and accelerates business workflows, particularly in industries such as finance, healthcare, and logistics.

4. **Translation Layer:** The translation layer uses Azure Text Translation to convert extracted text into the desired target language. It supports over 100 languages, enabling businesses to operate across global markets with ease. Advanced AI ensures that the context, tone, and structure of the original text are preserved during translation, producing outputs that are both accurate and culturally appropriate. This layer also supports custom glossaries, which allow users to specify industry-specific terminology, ensuring consistency in technical translations. Whether for real-time needs or batch processing, the translation layer adapts to varying workloads and delivers high-quality outputs tailored to user requirements.
5. **Storage Layer:** The storage layer consists of Azure Blob Storage for handling document storage and for managing metadata. Azure Blob Storage ensures secure storage of original files, intermediate outputs, and translated results. Meanwhile, the database maintains critical metadata, such as processing status,

user details, and logs, facilitating efficient tracking and retrieval of documents throughout the workflow.

6. **Notification and User Interaction:** This layer enhances user experience by keeping users informed about the status of their tasks. Azure Communication Services send email or SMS notifications when documents are translated and ready for download. The web application interface allows users to monitor progress, interact with the system, and retrieve completed translations, providing a seamless and intuitive experience.

7. **Monitoring and Analytics:** Power BI integration offers analytics dashboards for tracking usage trends, enabling data-driven decisions and optimization of the workflow.

4. Data management

4.1 Input Data Handling

Data Integration:

Users upload files through a web interface or APIs. Uploaded documents are validated for:

- File format compatibility (e.g., .pdf, .jpg, .png).
- File size limits (e.g., ≤ 40 MB).

Storage:

Uploaded documents are temporarily stored in **Azure Blob Storage** under a raw-documents container. Metadata such as file name, size, upload timestamp, and user ID is recorded in **Postgres SQL Database** for tracking.

4.2 Intermediate Data Management

4.2.1 Text Extraction:

Azure OCR processes the uploaded files and generates structured JSON outputs containing:

- Extracted text.
- Layout information, including bounding boxes and text regions.

4.2.2 Structured Information:

The JSON output is stored temporarily in an ocr-output container in Azure Blob Storage. Metadata, such as the job ID, processing time, and language detected, is added to the database for traceability.

4.2.3 Data Processing:

Azure Document Intelligence takes OCR outputs and identifies:

- Key-value pairs.
- Tables and structured entities (e.g., invoice numbers, customer names). The processed data is saved in structured formats like JSON or CSV for further use.

4.3 Translation Data Management

4.3.1 Translation Input:

Extracted or structured text from the previous stages is passed to Azure Text

Translation. Inputs include:

- Source language text.
- Target language.
- Optional custom glossary for domain-specific translations.

4.3.2 Translation output

Translated text is saved in the translated-documents container in Azure Blob Storage. Additional metadata, such as source-target language pairs and translation status, is stored in the database.

4.4 Metadata Management

4.4.1 Database Usage:

Postresql stores critical metadata for each document, including:

- File details: name, size, and upload date.
- Processing status: "Uploaded," "Processing," "Translated."
- Links to Blob Storage locations for intermediate and final outputs.

Tracking:
This metadata supports querying for document status, user activity logs, and audit trails.

- for GDPR compliance, and users confirm data privacy before processing.

4.5 Notifications and Delivery

4.5.1 Completion Notifications:

Once the processing of a document is complete, users are notified through **Azure Communication Services**, which delivers timely updates via email or SMS. The notifications include a secure, time-limited download link to the translated document, ensuring that only authorized users can access the file. Along with the download link, metadata such as the original file name, the translation's target language, and the processing status (e.g., "Completed" or "Failed") are included to provide a clear summary of the translation job. These notifications enhance user convenience by eliminating the need for manual checks on the translation status.

- A secure download link to the translated document.
- Metadata such as the file name and processing status.

4.5.2 Final Outputs:

The translated documents are stored temporarily and are available for download within a specific retention window. This ensures that users have ample time to access their files, even if immediate retrieval is not possible. After the retention period lapses, documents are automatically archived or securely deleted based on organizational data retention policies. Archiving options allow for long-term storage in Azure Blob Storage's cool or

5. Folders and code modules

5.1 React Frontend folder

The frontend provides a user interface for uploading documents, monitoring translation progress, and retrieving results. It leverages React for component-based architecture, TypeScript for type safety, and Tailwind CSS for styling.

1. **components/**
    - **Purpose**: Encapsulates reusable UI components for specific features.
    - **Examples**:
        - **Upload/**: Handles file selection and upload. Components like FileUpload.tsx allow users to upload documents, while ProgressBar.tsx visually tracks upload and processing progress.
        - **Translation/**: Displays translation results and processing status using components like TranslationStatus.tsx and TranslationResult.tsx.
        - **Navbar/**: Provides a consistent navigation bar across the app.
2. **pages/**
    - **Purpose**: Contains top-level components for routing purposes.
    - **Examples**:
        - Home.tsx: A landing page where users can upload documents and view translation status.
        - Dashboard.tsx: Displays a history of uploaded documents and their statuses.
        - Login.tsx: Handles user authentication via SSO or credentials.
3. **services/**
    - **Purpose**: Manages API communication with the backend.
    - **Examples**:
        - documentService.ts: Includes methods for uploading documents, checking status, and fetching results.
        - translationService.ts: Handles translation-specific API requests, such as fetching translated content.

4. **hooks/**
    - **Purpose**: Contains custom React hooks to encapsulate reusable logic.
    - **Examples**:
        - useAuth.ts: Manages user authentication state and token handling.
        - useFetch.ts: Simplifies API calls and error handling.
5. **Styling with Tailwind CSS**:
    - **Purpose**: Provides utility-based styling for rapid development.
    - **Examples**:
        - Global styles are configured in index.css and tailwind.config.js.
        - Component-specific styles are applied using Tailwind classes directly in JSX or via module CSS.

5.2 Backend folder

The backend handles document storage, processing, and integration with external services like Azure OCR and Text Translation. FastAPI provides a scalable, asynchronous API layer, while PostgreSQL manages persistent data storage.

1. **api/**
    - **Purpose**: Defines API routes and endpoints for document processing and translation.
    - **Examples**:
        - **authentication.py**: Handles user login and token generation.
        - **documents.py**: Manages file uploads, storage in Azure Blob Storage, and retrieval of document details.
        - **translation.py**: Integrates with Azure Text Translation for translating document content.
2. **core/**
    - **Purpose**: Contains core application logic and configuration.
    - **Examples**:

    - **config.py**: Reads environment variables (e.g., database URLs, Azure keys) for centralized configuration.
    - **security.py**: Implements authentication logic, such as OAuth2 or JWTz

3. **db/**
    - **Purpose**: Manages database models and interactions using SQLAlchemy.
    - **Examples**:
        - **models.py**: Defines database schemas, such as User, Document, and TranslationJob.
        - **session.py**: Handles database connection pooling and session management.
4. **services/**
    - **Purpose**: Implements business logic and external integrations.
    - **Examples**:
        - **azure_ocr.py**: Communicates with Azure OCR to extract text from uploaded documents.
        - **azure_translation.py**: Calls Azure Text Translation for converting extracted text into the desired language.
        - **document_processing.py**: Coordinates workflows for document processing, including storage and translation.
5. **main.py**
    - **Purpose**: Entry point for the FastAPI application.
    - **Example**: Initializes the app, configures middleware (e.g., CORS), and registers API routes.
6. **tests/**
    - **Purpose**: Contains unit and integration tests to ensure functionality.
    - **Examples**:
        - **test_documents.py**: Tests document upload, storage, and retrieval logic.
        - **test_translation.py**: Verifies integration with Azure Text Translation.

6. Example outputs

This section entails few reference outputs.

The images below demonstrate a document successfully converted from Swedish to English. The translation preserves the original layout, formatting, and structure, ensuring that the translated version maintains consistency and readability

Skapa flytt

Använd dessa instruktioner för att skapa en in- eller utflytt av kund, eller både och samtidigt.

*Det går även att skapa en flytt via noden **Flyttar**, och då anger man istället den aktuella anläggningen inuti skärmen.*

1. Gå in på noden **Anläggningar** och sök upp den anläggning som skall få en flytt.
2. Högerklicka på anläggningen (eller använd menyn Verktyg) och välj **Skapa flytt**, alternativt genom knappen Operationer ifall du har öppnat anläggningen.
3. Fönstret **Skapa flytt** öppnas nu.
4. Tillgänglig information från anläggningen och tillhörande aktivt fakturaställe kommer nu att vara förifyllt i flytten. Mer information kan nu läggas till eller ändras i tillgängliga kolumner.
5. Nu kan användaren trycka OK så skapas en utflytt som systemet kan behandla för att skapa Prodat, uppdatera information på fakturaställe etc.
6. Default-inställningen i ProBill är att endast utflytt är aktiv i rutan vid skapande av ny Flytt.
7. Om det endast ska registreras en inflytt så kan kolumnen **Utflytt** under **Egenskaper** sättas till NEJ och kolumnen **Inflytt** till JA. Värdena i rutan för inflytt måste då fyllas i av användaren.

Sida 2 av 7

Create move

Use these instructions to create a move-in or out-of-customer, or both at the same time.

It is also possible to create a move via the Moves node, and then you enter the current facility inside the screen instead.

1. Go to the node **Facilities** and find the facility that is to be moved.
2. Right-click on the facility (or use the Tools menu) and select **Create Move,** or through the Operations button if you have opened the facility.
3. The **Create Move window** will now open.
4. Available information from the facility and associated active invoice point will now be pre-filled in the move. More information can now be added or changed in the available columns.
5. Now the user can press OK and a move out is created that the system can process to create Prodat, update information on the invoice site, etc.
6. The default setting in ProBill is that only Moving out is active in the box when creating a new Move.
7. If only one move-in is to be registered, the column **Moving out** under **Properties** can be set to NO and the column **Moving in** to YES. The values in the move-in box must then be filled in by the user.

Output: The output of the document translation system includes the translated document in the same format as the original (e.g., .docx, .pdf), ensuring the preservation of layout and formatting

7. Cloud Infrastructure

The document translation system's cloud infrastructure leverages Azure services to ensure scalability, security, and efficient workflow

7.1 Compute

1. **AWS EC2**

- **Purpose**: Hosts the backend (FastAPI) application to handle API requests for document upload, translation, and status tracking.
- **Features**:
    - Auto-scaling to handle increased user traffic.
    - Secure integration with Azure AD for user authentication.

2. **Github Actions**

- **Purpose**: Hosts the React-based frontend application for user interactions.
- **Features**:
    - Global content delivery for low-latency user experiences.
    - Continuous integration with GitHub/DevOps pipelines.

7.2 Storage

1. **Azure Blob Storage**

- **Purpose**: Stores documents and translations securely.
- **Structure**:
    - **Containers**:
        - raw-documents: For uploaded files.
        - ocr-output: For text extracted by Azure OCR.
        - translated-documents: For final translated files.

- **Features**:
    - Encryption at rest and during transit.
    - Metadata tracking for document processing.

7.3 Networking

1. **AWS EC2**

- **Purpose**: Provides global load balancing and accelerates content delivery.
- **Features**:
    - Routes traffic to the nearest AWS Frankfurt.
    - Protects against DDoS attacks with integrated security features.

2. **Fast API Management**

- **Purpose**: Acts as the gateway for API requests, ensuring security and monitoring.
- **Features**:
    - Rate-limiting and request throttling.
    - Centralized logging and debugging.

3. **AWS Route 53**

Purpose:AWS Route 53 is a scalable DNS service that routes internet traffic to resources based on domain names and manages email routing (SMTP) through MX records.

Features:

- **DNS Management**: Manages domain name records (A, CNAME, MX, etc.) for routing traffic.
- **Email Routing**: Supports SMTP routing for domain-based email services.
- **Rate-Limiting & Throttling**: Helps prevent traffic overloads by limiting requests.
- **Centralized Logging**: Integrates with Amazon CloudWatch for logging and troubleshooting DNS queries.

8. Deployment process

8.1 Prerequisites

1. **GitHub Repository Configuration**:
    - Ensure a GitHub repository contains all code and workflows.
    - Folder structure:
        - **.github/workflows/**: Contains YAML files for GitHub Actions workflows.
        - Code directories for frontend, backend, and infrastructure configurations.
2. **Azure Resources**:
    - Create the required Azure resources:
        - Blob Storage.
        - AI Services (OCR, Translation).
3. **Environment Variables**:
    - Configure environment variables for Azure services, including:
        - Storage account credentials.
        - AI service keys and endpoints.
        - Database connection strings.

4. **Authentication**:
    - Use OAuth Proxy for secure authentication.
5. **Environment Variables in Azure**:
    - Add the following to App Services and Function Apps:
        - AZURE_BLOB_ACCOUNT
        - AZURE_AI_TRANSLATION_ENDPOINT
        - AZURE_AI_TRANSLATION_KEY
        - POSTGRES_DB_CONNECTION_STRING

8.2 Cloud Deployment Steps

1. **Infrastructure Setup**:
    - Run the frontend.yml and backend.yml to deploy required using predefined templates using CI/CD pipeline setup in github actions.
2. **Backend Deployment**:
    - Run backend.yml to deploy FastAPI code:
        - Configures database connection and storage account integration.
3. **Frontend Deployment**:
    - Run frontend.yml to deploy the React frontend:
        - Configures the connection to the backend APIs.
4. **Post-Deployment Verification**:
    - Test endpoints for backend APIs.
    - Verify frontend connectivity and functionality.
    - Check the status of Azure Functions and ensure successful triggers.

8.3 CI/CD Pipeline Using JFrog Artifactory and AWS ECR

1. Prerequisites

1. **JFrog Artifactory**:
    - Ensure an active instance of JFrog Artifactory is set up.

- Configure a repository for storing build artifacts (e.g., .jar, .zip) and Docker images.

2. **AWS ECR**:
    - Set up an AWS ECR repository for storing Docker images.
    - AWS CLI configured with appropriate credentials and permissions.
3. **Source Code Repository**:
    - A GitHub or GitLab repository containing the source code.
4. **CI/CD Tool**:
    - GitHub Actions, or GitLab CI/CD.
5. **Build Tool**:
    - Maven, Gradle, or a similar tool for building application artifacts.

2. Pipeline Workflow

Pipeline Stages:

1. **Source Code Checkout**:
    - Pull code from the version control repository.
2. **Build**:
    - Compile the code, run tests, and generate build artifacts (e.g., .jar, .war).
3. **Artifact Management**:
    - Push the generated artifacts to JFrog Artifactory.
4. **Docker Image Build**:
    - Create a Docker image for the application.
5. **Push Docker Image to ECR**:
    - Authenticate and push the Docker image to AWS EC2.
6. **Deploy**:
    - Deploy the image to a AWS EC2.

9. Debugging and troubleshooting

1. Source Code Checkout Issues

If the pipeline fails during the source code checkout stage, the issue is often related to repository access permissions or incorrect configuration of the branch or repository URL. To fix this, ensure that the CI/CD tool (e.g., Jenkins, GitHub Actions) has the necessary permissions and the correct branch or URL is specified. If using SSH keys or access tokens, verify their validity. For example, in GitHub Actions, use the actions/checkout@v3 plugin with the correct branch reference.

2. Build Failures

Build errors typically stem from issues like compilation errors, missing dependencies, or misconfigured build tools. Review the build logs to identify specific errors and ensure that the correct tool version is installed on the build agent. Set up required environment variables like JAVA_HOME for Java-based builds. For Maven projects, use commands like mvn clean install to compile and package the application while skipping tests if necessary.

3. Artifact Upload to JFrog Artifactory

Artifact upload failures to JFrog Artifactory are often due to misconfigured credentials or repository access issues. Verify that the JFrog CLI is properly set up and the Artifactory credentials are correct. Ensure that the repository path is accurate and has appropriate permissions. Use the command jfrog rt upload to upload artifacts while specifying the repository path and target directory.

4. Docker Image Build Failures

Errors during the Docker image build process may occur due to incorrect Dockerfile syntax, missing files in the build context, or inaccessible base images. To troubleshoot, ensure that the Dockerfile follows proper syntax and all required files are included in the context. Use the docker build --progress=plain option to get verbose logs, which can help pinpoint the issue.

5. Docker Push to AWS ECR

Authentication errors or permission issues often prevent successful image pushes to AWS ECR. Ensure that the AWS CLI is configured with valid credentials and that the ECR repository exists. IAM policies attached to the CI/CD role must include permissions for pushing images to ECR. Use the aws ecr get-login-password command to authenticate, then tag and push the image using docker push.

6. Deployment Errors

Deployment failures in Kubernetes or AWS ECS are usually caused by misconfigured deployment files, insufficient node capacity, or access issues. Validate the deployment files to ensure proper syntax and required fields. Use commands like kubectl describe pods and kubectl logs to debug Kubernetes deployments. For AWS ECS, ensure the task definitions are correctly configured and that the cluster is healthy.

7. Logging and Monitoring

Logs are crucial for identifying and resolving pipeline issues. Review logs in the CI/CD platform (e.g., Jenkins, GitHub Actions) for failed stages. Check JFrog Artifactory logs for upload/download issues and AWS CloudTrail logs for ECR access details. Application logs captured via frameworks like Logback for Java can provide additional runtime insights.

8. Common Fixes

Network connectivity errors can be resolved by testing endpoints using tools like ping and setting up private network endpoints for secure communication. For credential-related issues, rotate API keys and store them securely in CI/CD secrets. If resource limits are

exceeded, increase memory and CPU allocations for build agents and optimize Docker images by reducing unnecessary layers.

10. Monitoring and Maintenance

The document translation system includes robust monitoring and maintenance features to ensure its reliability and efficiency. These features cover progress tracking, notifications, error handling, and system optimization.

1. Tracking Translation Progress

The system provides real-time tracking of translation tasks through a progress bar, allowing users to monitor the upload and translation processes. After uploading, file statuses are displayed, such as "Processing," "Translated," or "Ready for Download." Users can also access the "Translated Docs" section to view the history of previously translated files. This section includes details such as file name and status, helping users stay informed about the progress of their requests. To enhance this feature, real-time updates can be integrated using backend API calls.

2. Email Notifications

To keep users informed, the system sends email notifications upon the completion of translation tasks. These emails include a subject line ("Translated Document is Ready for Download") and a link to access the translated files. The notifications ensure that users are promptly updated, even if they are not actively monitoring the application. Implementing email delivery monitoring tools such as Azure Communication Services or SendGrid can ensure that notifications reach users without delays or errors.

3. Error Handling and Logs

The system validates uploaded files to ensure they meet size and format requirements. Files exceeding 40 MB or unsupported formats are rejected, and users are provided with appropriate error messages. Logs are generated for failed operations, such as upload errors or translation failures, to facilitate troubleshooting. Administrators can review these logs to identify and resolve recurring issues effectively.

4. Resource Optimization and Capacity Planning

Regular monitoring of the system's resource usage is critical to maintaining performance. For example, Azure Monitor can track metrics such as CPU and memory utilization during peak usage. Periodic maintenance, including clearing unused logs and managing storage limits, prevents system overloads. The glossary feature, which supports up to 2000 word pairs, requires careful management to avoid failures during the translation process.

By implementing these monitoring and maintenance practices, the document translation system ensures a seamless user experience while maintaining operational efficiency. Regular updates and proactive error handling further enhance system reliability.

10.1 Azure cloud costing

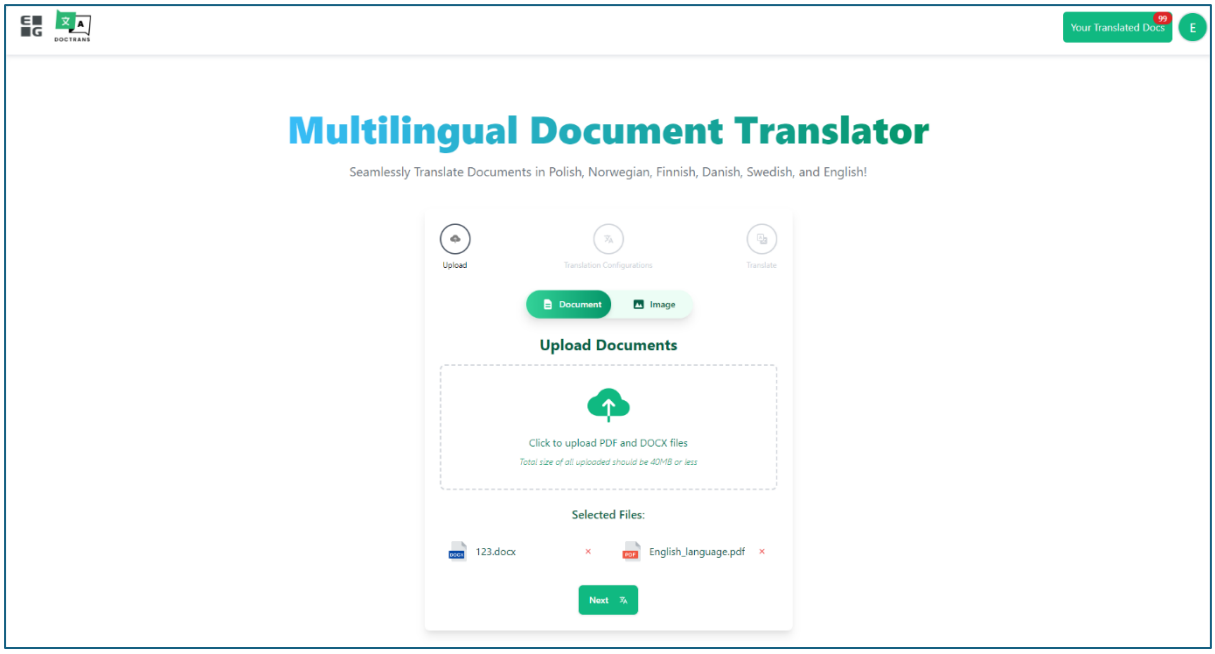
We have developed a Cloud Cost Calculator to help you assess your usage costs based on your specific needs and configurations. The calculator presents a general estimation based on the three tiers and includes a breakdown of assumptions used in the calculation. You can customize the values based on your number of documents, number of pages per document, number of characters per page, number of images per document. The cost assumptions made during our calculation are available in the workbook, allowing you to review or modify them as needed. The below attached screenshot is of the calculator.

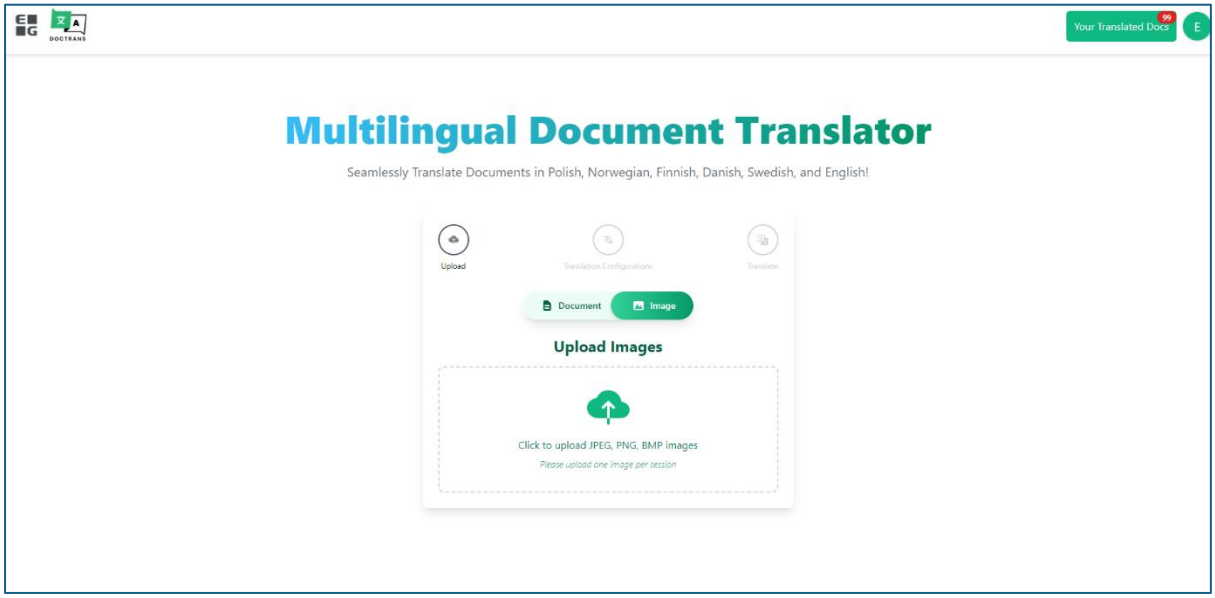
Link : EG AI DocTrans Cost Calculator.xlsx

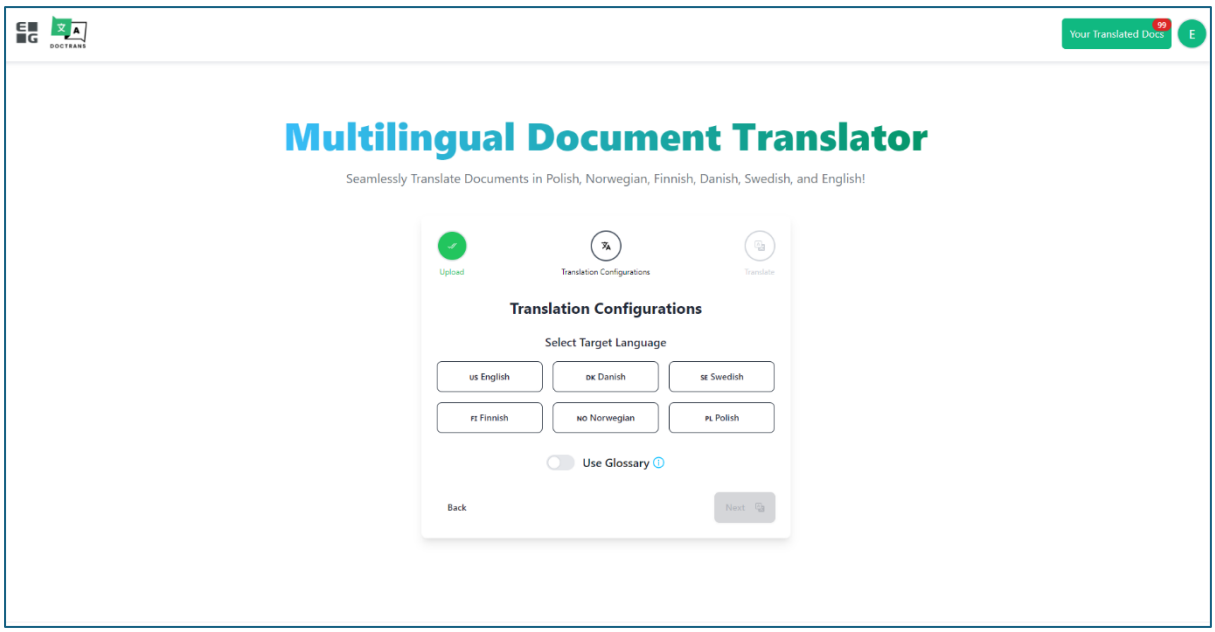
PS: This is a cost calculator to give an estimate figure and may vary case to case basis with varying requirements & configurations.

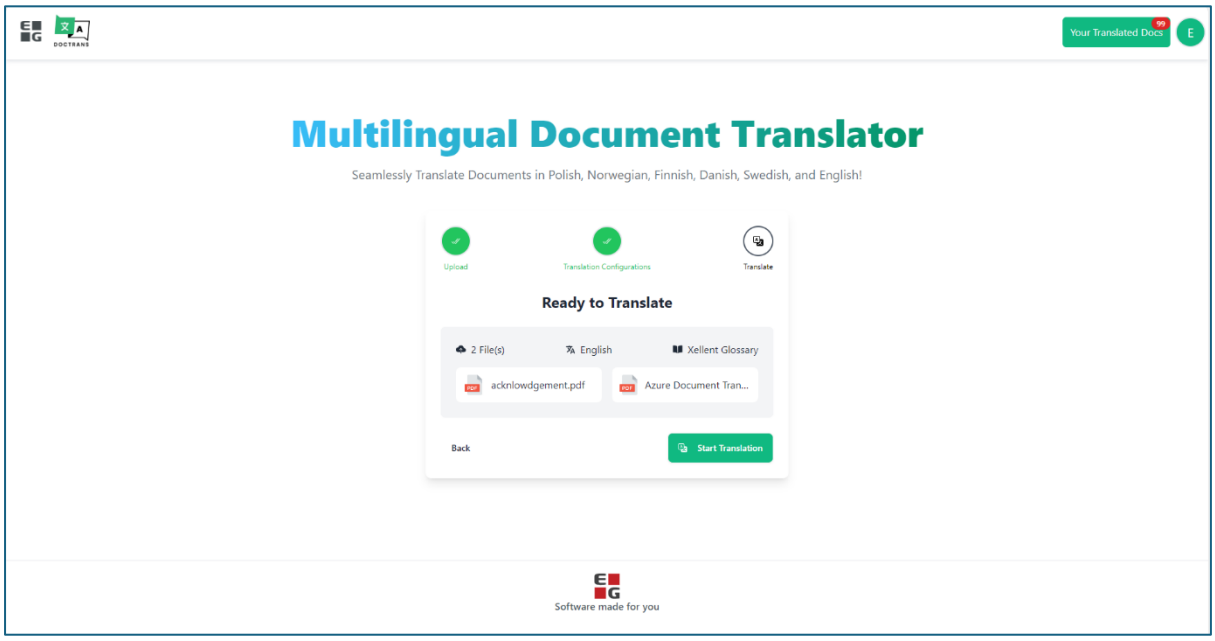
11. UI Wireframe

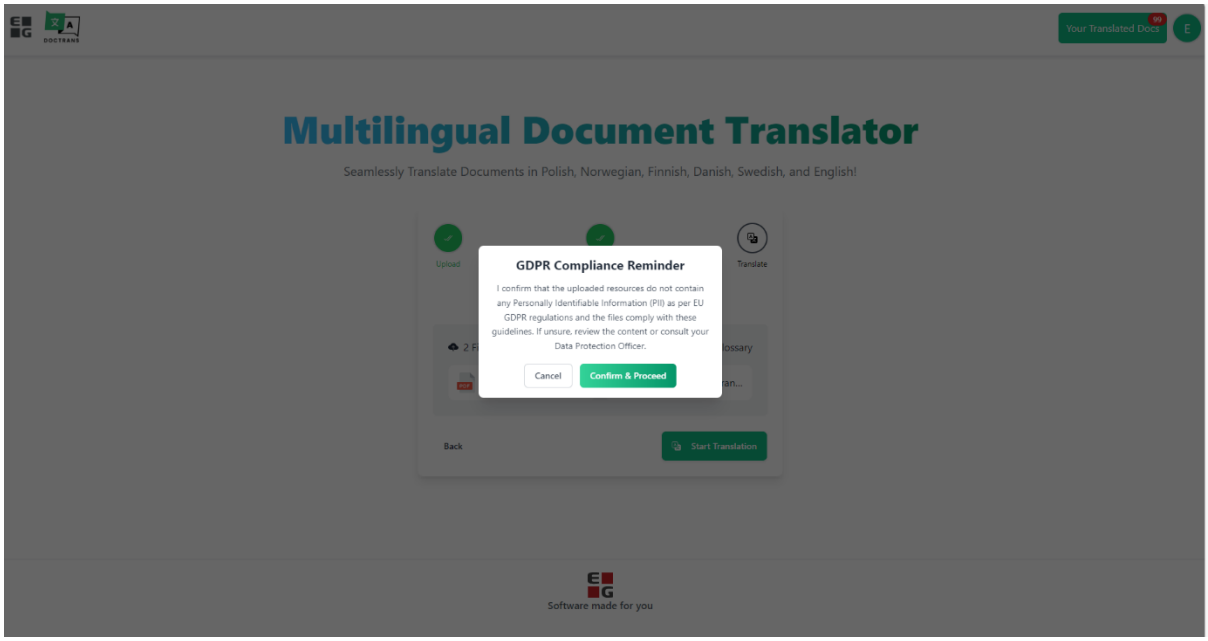
11.1 User Flow:

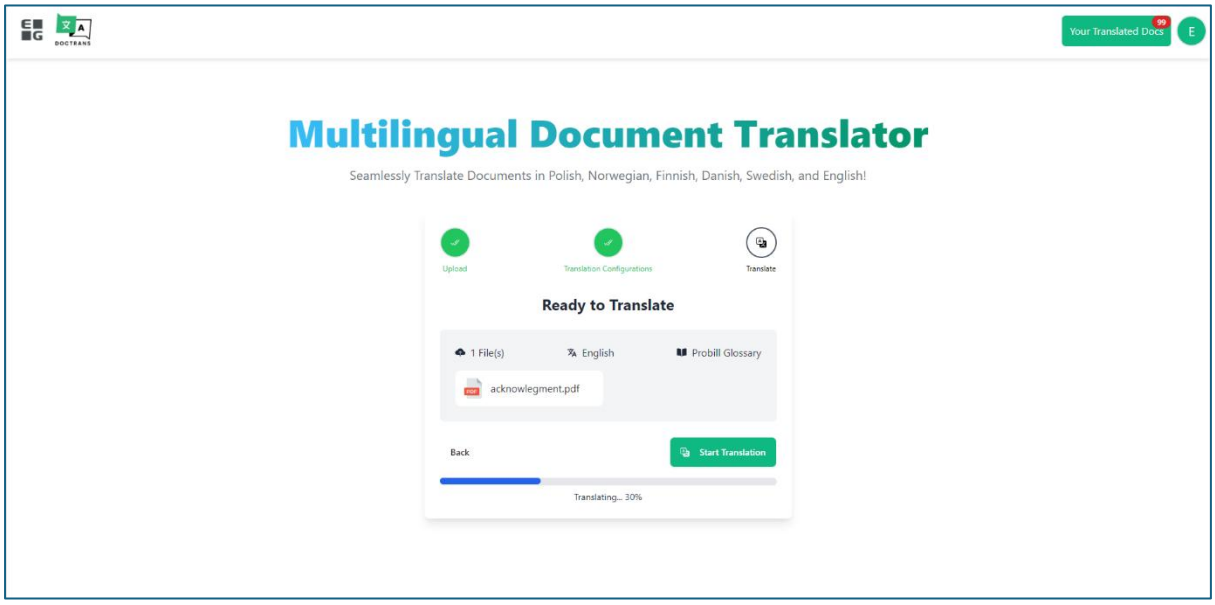





Your Translated Docs
99
E
Multilingual Document Translator
Seamlessly Translate Documents in Polish, Norwegian, Finnish, Danish, Swedish, and English!
Upload
Translation Configurations
Translate
Translation Configurations
Select Target Language
US English
DK Danish
SE Swedish
FI Finnish
NO Norwegian
PL Polish
Use Glossary
Back
Next


Your Translated Docs
99
E
Multilingual Document Translator
Seamlessly Translate Documents in Polish, Norwegian, Finnish, Danish, Swedish, and English!
Upload
Translation Configurations
Translate
Ready to Translate
2 File(s)
English
Xellent Glossary
acknowdgement.pdf
Azure Document Tran...
Back
Start Translation
Software made for you





12. Deployed links

1. GitHub link – https://github.com/EG-A-S/EG_Document_translation.git
2. Functional documentation link – Functional_Document_DOCTRANS.docx
3. Handover documentation SharePoint link – EG AI DOCTRANS DOCUMENTATION

13. Contact information

Please reach out to below to share your thoughts or for any queries

- Jaroslaw Krochmalski (xxhoz@eg.dk)
- Prakhyath Rai (prrai@eg.dk)